TUSAŞ LIFT UP · TEKNİK YOL HARİTASI

Lagrange + EMC: başlıktaki yöntemi demoya kazandırmak

Demodaki ana eksik nedir, literatür bunu nasıl çözüyor, bizim kodumuzda adım adım nasıl yapılır?

1. Sorun

Proje başlığı "**Lagrange gevşetmesi ile EMC duyarlı** otomatik rotalama" diyor. Demoda ise bu iki özellik ayrı yöntemlerde duruyor:

Yöntem	Lagrange var mı?	EMC'yi sağlıyor mu?	Alt sınır veriyor mu?
d) Lagrange	Var, ama **yalnızca kapasite** kısıtında	**Hayır (895 ihlal noktası)**	Evet (yalnızca kapasite modeli için)
e) / f) bütünleşik	Yok: sabit ceza + söküp yeniden rotalama (sezgisel)	**Evet (0 ihlal)**	Hayır
g) Lagrange + EMC (hedef)	Kapasite **ve** EMC çarpanlarla gevşetilir	**Evet (onarım adımıyla 0 ihlal)**	**Evet (EMC dahil modelin alt sınırı)**

Bu iş **LIFT UP kapsamı içindedir**: yeni bir konu açmaz, demodaki d) ve e) yöntemlerini birleştiren yeni bir yöntem ekler. Danışmanın da büyük ihtimalle ilk soracağı şey budur.

2. Hedef ve başarı ölçütleri

Yeni yöntem **g) Lagrange + EMC**, tek çalıştırmada hem uygun bir çözüm (üst sınır) hem de EMC'yi içeren modelin alt sınırını üretmeli.

#	Ölçüt	Nasıl ölçülür?
B1	Ana senaryoda beş bağımsız denetimin hepsi 0	`checks.py` (değişmez)
B2	Sağlamlık testinde 40/40 temiz	`benchmark.py` 'ye g) eklenir
B3	**Alt sınır geçerli:** küçük örnekte alt sınır ≤ kesin MILP optimumu	`exact.py` ile otomatik test
B4	Optimallik farkı raporlanır; hedef: e)'nin %6,5'lik farkından küçük	(üst − alt) / alt
B5	Toplam uzunluk e)'den kötü değil (≈ 66,05 m)	`metrics.md`

Danışmana nasıl anlatılır?

"Demoda Lagrange gevşetmesi şu an yalnızca kapasite kısıtında; EMC'yi sağlayan yöntemimiz ise sezgisel. Ana hedefimiz ikisini birleştirmek: EMC ayrımını 'şu iki ayrıt birlikte kullanılamaz' kısıtları olarak yazıp kapasiteyle birlikte Lagrange çarpanlarıyla gevşetmek, Lagrange çözümünü mevcut sezgisel yöntemimizle onarmak. Böylece aynı yöntemden hem kurallara uyan bir çözüm hem de 'en iyi çözümden en fazla şu kadar uzağız' diyen bir alt sınır elde edeceğiz. Bu yaklaşım çip tasarımındaki crosstalk kısıtlı rotalamada kullanılan yöntemlerle aynı aileden."

3. Literatür bize ne söylüyor?

"EMC ayrımı + Lagrange" kombinasyonu havacılık kablo literatüründe az çalışılmış; ama **çip tasarımında (VLSI)** neredeyse aynı problem 25 yıldır çözülüyor. Orada kablolar arasındaki parazite *crosstalk* denir ve ayrım mesafesi kısıtları vardır.

Zhou ve Wong — crosstalk kısıtlı rotalama[1,2]

Crosstalk kısıtlı labirent rotalamasının çok kısıtlı en kısa yol problemine denk ve güçlü NP-tam olduğunu gösterir; Lagrange gevşetmesine dayalı bir sezgisel önerir (1997). 1999'daki çalışmada Steiner ağacı formülasyonu + Lagrange gevşetmesiyle global rotalama yapılır; crosstalk sınırını aşan ağlar Lagrange ile yeniden rotalanır. **Bizim EMC problemimizin en yakın karşılığı.**

PathFinder — müzakereli tıkanıklık[3]

FPGA rotalamasının standardı. Her sinyal tek tek en kısa yolla rotalanır; paylaşılan kaynakların maliyeti talebe göre her turda artırılır, sinyaller "pazarlık" ederek ayrışır. **Demodaki e) yöntemi bu fikre çok benzer**: büyüyen EMC cezası aslında öğrenilen bir fiyattır. Eksik olan, bu fiyatı Lagrange çarpanı olarak kurup alt sınır elde etmektir.

Carrabs ve Gaudioso — çakışan ayrıt çiftleri[4]

"Şu iki ayrıt birlikte kullanılamaz" kısıtlı en küçük kapsayan ağaç problemi. Çakışma kısıtları Lagrange ile gevşetilir; alt sınır için dual ascent + subgradient, üst sınır için alt sınır hesabında karşılaşılan uygun olmayan çözümlerden **çakışmaları kaldıran bir sezgisel** kullanılır. **Bizim EMC kısıtımız tam olarak bu biçimde**: farklı sınıftan iki kablo, birbirine yakın iki ayrıtı birlikte kullanamaz.

Lucena — relax-and-cut[5]

Kısıt sayısı çok büyükse hepsini baştan gevşetmek yerine, yalnızca **ihlal edilenleri** çalışma sırasında gevşetilen kümeye ekleyen Lagrange yöntemi. Bizde EMC çakışma çifti sayısı yüz binleri bulabilir; ama herhangi bir anda ihlal edilen çift sayısı küçüktür. Bu yüzden relax-and-cut doğal seçim.

Barahona ve Anbil — volume algoritması[6]

Klasik subgradient iyi alt sınır verir ama uygun bir çözüm vermez. Volume algoritması subgradient'i genişleterek alt sınırla birlikte **yaklaşık uygun bir çözüm** de üretir. Subgradient salınırsa ya da onarım zorlanırsa ikinci seçenek.

Fisher — Lagrange gevşetmesi[7]

Temel çerçeve: zor kısıtlar çarpanlarla amaca taşınır, kalan problem kolaydır ve her çarpan için bir alt sınır verir. Demodaki d) yöntemi zaten bu çerçevede.

Literatürden çıkan reçete

(1) EMC'yi **ayrıt çifti çakışma kısıtları** olarak yaz, (2) kapasiteyle birlikte **Lagrange çarpanlarıyla gevşet** (yalnızca ihlal edilen çiftleri, relax-and-cut ile), (3) her yinelemede alt problemi **kablo başına bükülmeyi bilen en kısa yol** olarak çöz, (4) Lagrange çözümünü **çakışma kaldıran bir onarım sezgiseliyle** uygun hale getir. Onarım için demodaki e) yöntemi zaten hazır.

4. Matematik modeli

4.1 EMC kısıtını yazmak

İki ayrıt *e* ve *f*'nin doğru parçaları arasındaki en kısa mesafe, kablo sınıfları *c* ve *c′* için gereken ayrım mesafesinden küçükse, farklı sınıftan iki kablo bu iki ayrıtı birlikte kullanamaz:

```
x[k,e] + x[m,f] ≤ 1      her k ∈ c, m ∈ c' (c ≠ c'),  d(e,f) < s(c,c')
```

Bu, demodaki kesin MILP modelinin (`exact.py`) zaten kullandığı biçimdir; çakışan ayrıt çiftlerini bulan `_close_edge_pairs` fonksiyonu hazırdır (yalnızca farklı ayrıtları döndürür; iki kablonun **aynı** ayrıtı kullanması, e = f, ayrıca eklenmelidir). Bağımsız EMC denetimi de mesafeyi doğru parçalarına kesin olarak ölçtüğünden, model ile denetim aynı kuralı konuşur.

4.2 Gevşetme

Kapasite kısıtı için $\lambda_e \geq 0$ (demoda var), her EMC çakışma kısıtı için $\mu_{k,e,m,f} \geq 0$ (yeni). Gevşetilmiş problemde her kablo kendi en kısa yolunu şu maliyetle bulur:

```
maliyet_k(e) = uzunluk(e)  +  l_k · λ_e  +  Σ  μ_(k,e,m,f)          (k'nın e ayrıtını içeren aktif
çakışma kısıtları)
L(λ, μ)      = Σ_k  EnKısaYol_k(maliyet_k, dönüş çizgesinde)  −  K · Σ_e λ_e  −  Σ μ
```

Her λ, μ ≥ 0 için L(λ, μ), **bükülme + kapasite + EMC** modelinin optimumuna bir alt sınırdır. Bükülme sert kısıt olarak dönüş çizgesinde kalır (demodaki `turn_aware_path`), gevşetilmez.

4.3 Çarpanların güncellenmesi (d'deki döngü + μ bloğu)

```
g_λ(e)        = Σ_k l_k · x[k,e]  −  K                    → λ_e  ← max(0, λ_e + t · g_λ(e))
g_μ(k,e,m,f)  = x[k,e] + x[m,f]  −  1                     → μ    ← max(0, μ + t · g_μ)
t (Polyak)    = θ · (üst_sınır − L) / ‖g‖²
```

4.4 Relax-and-cut: çok sayıda kısıtla baş etmek

Çakışma kısıtı sayısı (kablo çifti × yakın ayrıt çifti) çok büyüktür. Bu yüzden μ'ler yalnızca **aktif küme** için tutulur: bir yinelemede ihlal edilen (iki kablonun da kullandığı) çift aktif kümeye eklenir; çarpanı uzun süre 0'da kalan çift kümeden çıkarılabilir[5]. Kümede olmayan kısıtların çarpanı 0 kabul edildiğinden alt sınır geçerliliğini korur.

4.5 Uygun çözüm (üst sınır): onarım

Lagrange yolları genellikle "neredeyse uygundur"; birkaç çakışma kalır. Her birkaç yinelemede bir:

1. Lagrange maliyetleri (`uzunluk + λ + μ`) e) yönteminin **temel maliyeti** olarak verilir. Böylece e)'nin demetleme indirimi, EMC cezası ve kapasite cezası, Lagrange'ın "öğrendiği" fiyatların üzerine eklenir.
2. Lagrange yolları başlangıç çözümü olarak alınır; yalnızca çakışan kablolar söküp yeniden rotalanır (e)'nin mevcut mantığı).
3. Bağımsız denetimlerden temiz geçen en kısa çözüm üst sınır olarak saklanır.

5. Algoritma

```
girdi: çizge G, dönüş çizgesi T, kablolar, ayrım tablosu s, kapasite K
çakışma_çiftleri ← _close_edge_pairs(G, s)          # sınıf çifti başına, bir kez (exact.py'den)
λ ← 0;  μ ← {} (aktif küme boş);  alt ← −∞;  üst ← +∞

tekrarla (en çok N yineleme):
    her kablo k için:
        maliyet_k ← uzunluk + l_k·λ + Σ aktif μ'ler (k'nın ayrıtlarına)
        P_k ← turn_aware_path(T, maliyet_k, k.başlangıç, k.bitiş)        # bükülme sert
    L ← Σ_k maliyet_k(P_k) − K·Σλ − Σμ ;  alt ← max(alt, L)

    ihlaller ← {(k,e,m,f) : e ∈ P_k, f ∈ P_m, (e,f) çakışma çifti, sınıf(k) ≠ sınıf(m)}
    aktif kümeye ihlalleri ekle (relax-and-cut)

    her r yinelemede bir:                                              # onarım
        R ← route_integrated(..., temel_maliyet = uzunluk + λ + μ, başlangıç = {P_k})
        eğer R bağımsız denetimlerden temizse:  üst ← min(üst, uzunluk(R))

    alt sınır p kez iyileşmediyse θ ← θ / 2
    λ, μ ← izdüşümlü subgradient adımı (Polyak, hedef = üst)
    dur: (üst − alt)/alt < ε  veya  ‖g‖ = 0  veya  süre sınırı

çıktı: en iyi uygun çözüm R*, alt sınır, fark, yineleme geçmişi
```

Üç seçenek: neden B?

Seçenek	Ne yapar?	Alt sınır	İş yükü
A) Ceza → çarpan	e)'deki sabit EMC cezasını, ihlale göre büyüyen ayrıt başına "fiyat"a çevirir (PathFinder tarzı)	**Yok**	Düşük
B) Tam Lagrange + relax-and-cut + onarım (önerilen)	Yukarıdaki algoritma	**Var**	Orta
C) Dantzig–Wolfe / sütun üretimi	Yol değişkenleriyle LP gevşetmesi; daha sıkı sınır	**Daha sıkı**	**Yüksek (dönem için riskli)**

A, B'nin ilk adımı olarak da görülebilir: önce fiyatları öğrenen bir döngü kurulur, sonra alt sınır hesabı eklenir. C ise yalnızca B'nin alt sınırı çok zayıf çıkarsa, karşılaştırma amaçlı düşünülmelidir.

Neden bu tasarım bizim projeye uygun?

- Hiçbir mevcut parça atılmaz: d)'nin subgradient döngüsü, e)'nin onarımı, dönüş çizgesi, `exact.py`'nin çakışma çiftleri ve bağımsız denetimler aynen kullanılır.
- Başlığın üç anahtar kelimesi tek yöntemde birleşir: **Lagrange** (alt sınır), **EMC** (gevşetilen çakışma kısıtı), **3B gövde** (dönüş çizgesinde bükülme).
- Sonuç ölçülebilir ve kanıtlanabilir: alt sınırın geçerliliği küçük örnekte kesin MILP ile otomatik test edilir.

6. Koddaki karşılığı

Dosya	Değişiklik
`harness_demo/routing/lagrangian_emc.py` yeni	`route_lagrangian_emc(...)` : yukarıdaki algoritma; `RoutingResult("lagrangian_emc", "g) Lagrange + EMC", ...)` ve yineleme geçmişi (alt/üst sınır, aktif küme boyutu, ihlal sayısı)
`harness_demo/exact.py`	`_close_edge_pairs` ve `segment_distances` ortak bir modüle (ör. `routing/conflicts.py`) taşınır; iki yöntem aynı çakışma tanımını kullanır
`harness_demo/routing/sequential.py`	`_SequentialConfig` 'e `base_cost` (temel maliyet) ve `route_integrated` 'e `initial_routes` (başlangıç çözümü) parametreleri; varsayılanları bugünkü davranışı aynen korur
`harness_demo/routing/lagrangian.py`	Değişmez (d yöntemi karşılaştırma için kalır); ortak subgradient adımı istenirse yardımcı fonksiyona alınır
`main.py` , `metrics.py` , `benchmark.py`	g) yöntemi tabloya, sağlamlık testine ve `lagrangian_history.json` 'a eklenir; alt sınır ve fark raporlanır
`visualize.py` , `viewer/`	Yakınsama grafiğine g) eğrisi; 3B sayfaya yedinci sekme
`tests/test_lagrangian_emc.py` yeni	Aşağıdaki testler

Yazılacak testler

- **Alt sınır geçerliliği:** `exact.py` 'nin küçük örneğinde (S4 + D2, 0,2 m mesh) g)'nin alt sınırı ≤ MILP'in kanıtlanmış alt sınırı ya da bulduğu çözüm.
- **Monotonluk:** kaydedilen en iyi alt sınır hiç azalmaz ve hiçbir zaman üst sınırı geçmez (d için var olan testin benzeri).
- **Uygunluk:** ana senaryoda g)'nin çözümü beş bağımsız denetimden temiz geçer; sağlamlık testinde 40/40.
- **EMC olmadan d)'ye indirgeme:** ayrım mesafeleri 0 yapılınca g)'nin alt sınırı d)'ninkine eşit (aynı kapasite modeli, bükülme hariç tutulursa).
- **Geriye uyumluluk:** `base_cost` / `initial_routes` verilmediğinde e) ve f) sonuçları bit düzeyinde aynı (regresyon tablosu değişmez).

7. Takvim (öneri)

Rehberdeki yol haritasının 3. iş paketine (Şubat – Nisan 2027) yerleşir. Danışman onayıyla daha erken başlanabilir.

Süre	Kilometre taşı	Çıktı / bitti sayılması için
Aralık 2026	M0 — Onay ve hazırlık	Danışmanla yaklaşımın konuşulması; Zhou–Wong ve Carrabs–Gaudioso makalelerinin okunması; `_close_edge_pairs` 'in ortak modüle taşınması (testler aynen geçer).
2 hafta	M1 — EMC çarpanları, düz çizge	Bükülme olmadan (düz çizgede) kapasite + EMC Lagrange; alt sınır küçük örnekte MILP'in altında (test B3).
1 hafta	M2 — Dönüş çizgesi	Alt problemler `turn_aware_path` ile; bükülme sert kısıt.
2 hafta	M3 — Onarım	`base_cost` ve `initial_routes` ile e)'ye bağlantı; ana senaryoda 0 ihlal (B1).
2 hafta	M4 — Relax-and-cut ve ayar	Aktif küme, adım boyu ayarı, durma ölçütleri; fark raporu (B4).
2 hafta	M5 — Deneyler	Sağlamlık testi (B2), mesh yakınsama, kesit alanı modu, e) ile karşılaştırma tablosu (B5).
1 hafta	M6 — Belgeleme	README, 3B sayfa sekmesi, bitirme raporunun yöntem bölümü.

8. Riskler ve önlemler

Risk	Önlem
Çakışma kısıtlarının Lagrange sınırı zayıf çıkabilir (bu tür kısıtların LP gevşetmesi bilinen bir zayıflıktır)	Sınıf düzeyinde toplulaştırılmış kısıtlar ("c sınıfı e'yi kullanıyorsa c' sınıfı N(e)'yi kullanamaz") denenir; fark dürüstçe raporlanır. Zayıf sınır bile e)'nin bugün hiç vermediği bir kanıttır.
Çakışma çifti sayısı bellek/süre sorunu yaratır	Çiftler sınıf çifti başına bir kez hesaplanır; μ yalnızca aktif küme için tutulur (relax-and-cut).
Subgradient salınır, alt sınır yavaş yükselir	Adım yarılama (d'de var), saptırılmış subgradient ya da volume algoritması[6].
Onarım uygun çözüm bulamaz	e)'nin yeniden başlatma mekanizması zaten var; en kötü durumda e)'nin çözümü üst sınır olarak kullanılır ve alt sınır yine geçerlidir.
Model (ayrıt çifti) ile denetim (örnek noktalar) farklı sonuç verir	İkisi de doğru parçalarına kesin mesafe kullanır; bir test, g)'nin "çakışmasız" dediği çözümde denetimin 0 bulduğunu doğrular.
Süre yetmez	M3 sonunda bile "alt sınır veren ve 0 ihlalli" bir yöntem elde edilmiş olur; M4–M5 iyileştirmedir.

Kaynakça

[1] H. Zhou, D. F. Wong, "Crosstalk-constrained maze routing based on Lagrangian relaxation," *Proc. IEEE International Conference on Computer Design (ICCD '97)*, Austin, TX, 1997, ss. 628–633. doi:10.1109/ICCD.1997.628931. kayıt

[2] H. Zhou, D. F. Wong, "Global routing with crosstalk constraints," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 18(11), 1683–1688, 1999. doi:10.1109/43.806813. (Konferans sürümü: DAC 1998, ss. 374–377.) dblp.org · scholars.hkbu.edu.hk

[3] L. McMurchie, C. Ebeling, "PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs," *Proc. ACM/IEEE International Symposium on Field-Programmable Gate Arrays (FPGA '95)*, 1995, ss. 111–117. doi:10.1145/201310.201328. ders notu (PDF)

[4] F. Carrabs, M. Gaudioso, "A Lagrangian approach for the minimum spanning tree problem with conflicting edge pairs," *Networks*, 78(1), 32–45, 2021. doi:10.1002/net.22009. dblp.org

[5] A. Lucena, "Non delayed relax-and-cut algorithms," *Annals of Operations Research*, 140(1), 375–410, 2005. doi:10.1007/s10479-005-3977-1. link.springer.com

[6] F. Barahona, R. Anbil, "The volume algorithm: producing primal solutions with a subgradient method," *Mathematical Programming*, 87(3), 385–399, 2000. dblp.org · optimization-online.org (PDF)

[7] M. L. Fisher, "The Lagrangian Relaxation Method for Solving Integer Programming Problems," *Management Science*, 27(1), 1–18, 1981. doi:10.1287/mnsc.27.1.1.

[8] T. Karlsson, E. Åblad, T. Hermansson, J. S. Carlson, G. Tenfält, "Automatic Cable Harness Layout Routing in a Customizable 3D Environment," *Computer-Aided Design*, 169, 103671, 2024. doi:10.1016/j.cad.2023.103671.

Kaynakların künye ve özetleri Kasım 2026'da doğrulanmıştır; tam metinler okunmamıştır. "PathFinder'ın e)'ye benzerliği" bizim yorumumuzdur; makalede Lagrange yorumu yapıldığı doğrulanmamıştır.